2. Analyze the two GenAI programs. The analysis must include:

a) Name of two GenAI programs
   1) Lumo and Leo

b) How you accessed them (e.g., Google query, CoPilot on my computer)
   1) Lumo on browser (accessed from protonmail)
   2) Leo from built-in

c) Prompt used. (You must use the same prompt for both GenAIs.)

Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):

• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt

d) Generated code from both GenAIs.

Leo:

```
// EECS 348 Assignment 4 - Sudoku Solver
// A recursive, object-oriented Sudoku solver using depth-first search with backtracking.
// Reads puzzle1.txt through puzzle5.txt from the current directory,
// prints each puzzle and all its solutions, then reports the count.
// Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
// Output: Puzzle grid, solution grid(s) or "No solution found", and solution count
// Author: Brave's Leo genAI
// Created: October 14, 2026
// Sources: Self-authored

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <iomanip>

using namespace std;

// Board stores the 9x9 grid. 0 means an empty cell.
class Board {
public:
    int grid[9][9];

    // Load the grid from a file. Each row has 9 tokens separated by whitespace.
    // '_' becomes 0; digits become their integer value.
    bool load(const string& filename) {
        ifstream infile(filename);
        if (!infile.is_open()) {
            cerr << "Could not open " << filename << endl;
            return false;
        }
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                string token;
                infile >> token;
                if (token == "_") {
                    grid[r][c] = 0;
                } else {
                    grid[r][c] = stoi(token);
                }
            }
        }
        infile.close();
        return true;
    }
```

```
    // Print the grid to stdout in a human-readable format.
    void print() const {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (c == 0) cout << "  ";
                if (c == 3 || c == 6) cout << " ";
                if (grid[r][c] == 0) {
                    cout << "_";
                } else {
                    cout << grid[r][c];
                }
                if (c < 8) cout << " ";
            }
            cout << endl;
        }
    }

    // Check whether placing value v at row r, column c is legal
    // under Sudoku rules (row, column, and 3x3 block uniqueness).
    bool isValid(int r, int c, int v) const {
        for (int i = 0; i < 9; i++) {
            if (grid[r][i] == v) return false;      // same row
            if (grid[i][c] == v) return false;      // same column
        }
        int br = (r / 3) * 3;
        int bc = (c / 3) * 3;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (grid[br + i][bc + j] == v) return false;  // same block
            }
        }
        return true;
    }
};

// Solver encapsulates the recursive backtracking search.
class Solver {
private:
    Board& board;           // reference to the puzzle board
    int solutionsFound;     // total number of solutions discovered

    // Return the next empty cell as [row, col], or [-1, -1] if full.
    vector<int> findEmpty() const {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (board.grid[r][c] == 0) {
                    return {r, c};
                }
            }
```

```
        }
        return {-1, -1};
    }

    // The recursive depth-first search.
    // When the board is completely filled (empty returns {-1,-1}) we've found
    // a solution and print it. Otherwise we pick the first empty cell, try
    // values 1-9, and recurse. If no value works we backtrack.
    void search() {
        vector<int> empty = findEmpty();
        int r = empty[0];
        int c = empty[1];

        // Board is full — we found a solution.
        if (r == -1) {
            solutionsFound++;
            board.print();
            cout << endl;
            return;
        }

        // Try each candidate value.
        for (int v = 1; v <= 9; v++) {
            if (board.isValid(r, c, v)) {
                board.grid[r][c] = v;   // place value
                search();               // recurse
                board.grid[r][c] = 0;   // backtrack: undo placement
            }
        }
    }

public:
    // Construct a solver bound to a specific board.
    explicit Solver(Board& b) : board(b), solutionsFound(0) {}

    // Run the solver. Returns the number of solutions found.
    int solve() {
        search();
        return solutionsFound;
    }
};

// Main: iterate over puzzle1.txt … puzzle5.txt, load each, solve, print results.
int main() {
    for (int p = 1; p <= 5; p++) {
        Board board;
        ostringstream oss;
        oss << "puzzle" << p << ".txt";
        string filename = oss.str();
```

```
        cout << "Puzzle file: " << filename << endl;
        cout << "Original puzzle:" << endl;

        if (!board.load(filename)) {
            cout << endl;
            continue;
        }

        board.print();

        Solver solver(board);
        int count = solver.solve();

        if (count == 0) {
            cout << "No solution found" << endl;
        }

        cout << "Solutions found: " << count << endl;
        cout << endl;
    }

    return 0;
}
```

Lumo:

```
/*******************************************************************************
 * EECS 348 - Software Engineering I
 * Assignment 4 - Recursive, Object-Oriented Sudoku Solver
 * ----------------------------------------------------------------------------
 * Program name : Assignment4.cpp
 * Description  : Reads Sudoku puzzle files (puzzle1.txt ... puzzle5.txt),
 *                prints each puzzle exactly as it was read, then finds EVERY
 *                solution using a depth-first search that backtracks. For each
 *                puzzle it prints the file name, the original puzzle, each
 *                solution found (or "No solution found"), and the total number
 *                of solutions.
 * Inputs       : puzzle1.txt ... puzzle5.txt in the current working directory.
 *                Each file supplies 81 cells; '1'-'9' are givens and '_'
 *                (also '.' or '0') marks a blank. Optional command-line
 *                arguments may name other files and/or "--max=N" to stop
 *                after N solutions per puzzle.
 * Output       : Console: file name, puzzle, all solutions, solution count,
 *                nodes searched, and elapsed time per puzzle.
 * Collaborators: <none>
 * Other sources: Skeleton generated with <GenAI #1 name> and <GenAI #2 name>;
 *                backtracking approach from the EECS 348 lecture slides.
 * Author       : Proton's Lumo genAI
 * Creation date: <2026-10-14>
 ******************************************************************************/

#include <array>
#include <chrono>
#include <cstddef>
#include <fstream>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>

// ---------------------------- shared constants ------------------------------
constexpr int kSize      = 9;              // grid is 9 x 9
constexpr int kBlockDim  = 3;              // each sub-grid is 3 x 3
constexpr int kCellCount = kSize * kSize;  // 81 cells per puzzle
constexpr unsigned short kAllDigits = 0x3FE;// bits 1..9 set (bit 0 unused)

// Map a (row, col) pair onto its 3x3 block number, 0..8.
inline int blockIndexOf(int row, int col) {
    return (row / kBlockDim) * kBlockDim + (col / kBlockDim);
}

// Build the one-hot bit mask for a digit 1..9.
inline unsigned short bitFor(int digit) {
    return static_cast<unsigned short>(1u << digit);
```

```
}

// Count the set bits of a mask (how many candidates remain).
inline int countBits(unsigned short mask) {
    int count = 0;
    while (mask != 0) {
        mask = static_cast<unsigned short>(mask & (mask - 1)); // clear lowest set bit
        ++count;
    }
    return count;
}

/*****************************************************************************
 * Class: SudokuGrid
 * Purpose: Value object holding the 81 cell values of one puzzle (0 = blank).
 *          Knows how to read itself from a file, print itself, and check that
 *          its given clues do not already contradict each other.
 * Authorship: <your name>, generated with <GenAI name> and revised by hand.
 *****************************************************************************/
class SudokuGrid {
public:
    // Start with an empty grid so a default-constructed object is always valid.
    SudokuGrid() { cells_.fill(0); }

    // Read 81 cells from a text file. Returns false if the file is missing or
    // does not contain 81 usable cells.
    bool loadFromFile(const std::string& fileName) {
        std::ifstream input(fileName.c_str());      // open the puzzle file
        if (!input.is_open()) {                     // report failure to caller
            return false;
        }
        cells_.fill(0);                             // reset any previous content
        int index = 0;                              // how many cells read so far
        char ch = '\0';                             // current character
        while (index < kCellCount && input.get(ch)) {// stop at 81 cells or EOF
            if (ch >= '1' && ch <= '9') {           // a given clue digit
                cells_[index++] = ch - '0';         // store its numeric value
            } else if (ch == '_' || ch == '.' || ch == '0') { // a blank cell
                cells_[index++] = 0;                // 0 marks "not yet filled"
            }
            // Anything else (spaces, tabs, newlines) is simply skipped.
        }
        return index == kCellCount;                 // success only if all 81 read
    }

    // Read-only accessor for one cell.
    int valueAt(int row, int col) const {
        return cells_[row * kSize + col];
    }
```

```
// Write accessor used by the solver while it tries and un-tries digits.
void setValue(int row, int col, int value) {
    cells_[row * kSize + col] = value;
}

// True when the cell has not been filled in yet.
bool isEmpty(int row, int col) const {
    return valueAt(row, col) == 0;
}

// Print the grid as nine rows; blanks appear as '_' so the original puzzle
// is still recognisable after solving.
void print(std::ostream& out) const {
    for (int row = 0; row < kSize; ++row) {          // one output line per row
        for (int col = 0; col < kSize; ++col) {      // nine cells per line
            const int value = valueAt(row, col);     // fetch this cell
            out << (value == 0 ? '_' : static_cast<char>('0' + value)) << ' ';
        }
        out << '\n';                                 // end of the row
    }
}

// Verify that no digit appears twice in any row, column, or 3x3 block of
// the *given* clues. A contradiction here proves no solution can exist.
bool hasConsistentGivens() const {
    for (int row = 0; row < kSize; ++row) {          // scan every row
        unsigned short seen = 0;                     // digits seen in this row
        for (int col = 0; col < kSize; ++col) {
            const int value = valueAt(row, col);     // current cell value
            if (value == 0) continue;                // blanks cannot clash
            const unsigned short bit = bitFor(value);// mask for this digit
            if ((seen & bit) != 0) return false;     // duplicate found
            seen = static_cast<unsigned short>(seen | bit); // record digit
        }
    }
    for (int col = 0; col < kSize; ++col) {          // scan every column
        unsigned short seen = 0;
        for (int row = 0; row < kSize; ++row) {
            const int value = valueAt(row, col);
            if (value == 0) continue;
            const unsigned short bit = bitFor(value);
            if ((seen & bit) != 0) return false;
            seen = static_cast<unsigned short>(seen | bit);
        }
    }
    for (int block = 0; block < kSize; ++block) {    // scan every 3x3 block
        unsigned short seen = 0;
        const int baseRow = (block / kBlockDim) * kBlockDim; // block origin
```

```
            const int baseCol = (block % kBlockDim) * kBlockDim;
            for (int r = 0; r < kBlockDim; ++r) {        // three rows in block
                for (int c = 0; c < kBlockDim; ++c) {    // three columns in block
                    const int value = valueAt(baseRow + r, baseCol + c);
                    if (value == 0) continue;
                    const unsigned short bit = bitFor(value);
                    if ((seen & bit) != 0) return false;
                    seen = static_cast<unsigned short>(seen | bit);
                }
            }
        }
        return true;                                // all clues agree
    }

private:
    std::array<int, kCellCount> cells_;   // row-major storage of the 81 cells
};

/*****************************************************************************
 * Class: CandidateTracker
 * Purpose: Keeps three arrays of bit masks describing which digits are already
 *          used in each row, column, and block. Candidates for a cell are then
 *          one bitwise operation instead of a nine-cell scan, which is the main
 *          reason the search finishes in milliseconds.
 * Authorship: <your name>, generated with <GenAI name> and revised by hand.
 *****************************************************************************/
class CandidateTracker {
public:
    // Build the masks from the clues already present in the grid.
    explicit CandidateTracker(const SudokuGrid& grid) {
        rowUsed_.fill(0);                   // no digits used anywhere yet
        colUsed_.fill(0);
        blockUsed_.fill(0);
        for (int row = 0; row < kSize; ++row) {         // walk the whole grid
            for (int col = 0; col < kSize; ++col) {
                const int value = grid.valueAt(row, col); // clue in this cell
                if (value != 0) {                   // only givens matter
                    markUsed(row, col, value);          // record the digit
                }
            }
        }
    }

    // Mask of digits that may legally be placed in (row, col).
    unsigned short candidatesAt(int row, int col) const {
        const unsigned short used = static_cast<unsigned short>(
            rowUsed_[row] | colUsed_[col] | blockUsed_[blockIndexOf(row, col)]);
        return static_cast<unsigned short>(kAllDigits & ~used); // invert + trim
    }
```

```
    // Record that a digit now occupies (row, col).
    void markUsed(int row, int col, int digit) {
        const unsigned short bit = bitFor(digit);        // one-hot mask
        rowUsed_[row] |= bit;                            // update its row
        colUsed_[col] |= bit;                            // update its column
        blockUsed_[blockIndexOf(row, col)] |= bit;       // update its block
    }

    // Undo markUsed when the search backtracks out of this cell.
    void markFree(int row, int col, int digit) {
        const unsigned short bit = static_cast<unsigned short>(~bitFor(digit));
        rowUsed_[row] &= bit;                            // clear from the row
        colUsed_[col] &= bit;                            // clear from the column
        blockUsed_[blockIndexOf(row, col)] &= bit;       // clear from the block
    }

private:
    std::array<unsigned short, kSize> rowUsed_;    // digits used per row
    std::array<unsigned short, kSize> colUsed_;    // digits used per column
    std::array<unsigned short, kSize> blockUsed_;  // digits used per 3x3 block
};

/*****************************************************************************
 * Class: SudokuSolver
 * Purpose: Owns the recursive depth-first search with backtracking. It mutates
 *          one working grid in place (restoring it on the way back up) and
 *          records a copy of the grid each time a complete solution is reached,
 *          so multiple solutions are reported.
 * Authorship: <your name>, generated with <GenAI name> and revised by hand.
 *****************************************************************************/
class SudokuSolver {
public:
    // solutionLimit == 0 means "report every solution" (the assignment default).
    SudokuSolver(const SudokuGrid& puzzle, std::size_t solutionLimit)
        : puzzle_(puzzle), limit_(solutionLimit), nodesVisited_(0), hitLimit_(false) {}

    // Run the search to completion (or until the optional limit is reached).
    void solve() {
        solutions_.clear();                      // fresh results
        nodesVisited_ = 0;                       // reset instrumentation
        hitLimit_ = false;
        if (!puzzle_.hasConsistentGivens()) {        // contradictory clues
            return;                              // provably no solution
        }
        SudokuGrid working(puzzle_);                 // mutable copy to search in
        CandidateTracker tracker(working);           // masks for that copy
        search(working, tracker);                    // start the recursion
    }
```

```
    const std::vector<SudokuGrid>& solutions() const { return solutions_; }
    std::size_t nodesVisited() const { return nodesVisited_; }
    bool hitLimit() const { return hitLimit_; }

private:
    // True once the caller-supplied cap on reported solutions is reached.
    bool limitReached() const {
        return limit_ != 0 && solutions_.size() >= limit_;
    }

    // One recursive step: pick the most constrained empty cell, try each of its
    // candidates, recurse, then undo the guess if the branch failed.
    void search(SudokuGrid& grid, CandidateTracker& tracker) {
        ++nodesVisited_;                        // count this visit
        if (limitReached()) {                   // optional early stop
            hitLimit_ = true;
            return;
        }

        int bestRow = -1, bestCol = -1;         // chosen empty cell
        int bestCount = kSize + 1;              // fewest candidates so far
        unsigned short bestMask = 0;            // candidates of that cell
        bool forced = false;                    // stop scanning if forced

        // Minimum-remaining-values scan: the cell with the fewest candidates is
        // tried first, which prunes the search tree dramatically.
        for (int row = 0; row < kSize && !forced; ++row) {
            for (int col = 0; col < kSize; ++col) {
                if (!grid.isEmpty(row, col)) continue;      // already filled
                const unsigned short mask = tracker.candidatesAt(row, col);
                const int count = countBits(mask);          // candidate count
                if (count == 0) {                           // dead end
                    return;                                 // backtrack now
                }
                if (count < bestCount) {                    // better cell
                    bestCount = count;
                    bestRow = row;
                    bestCol = col;
                    bestMask = mask;
                    if (count == 1) {                       // cannot do better
                        forced = true;
                        break;
                    }
                }
            }
        }

        if (bestRow < 0) {                      // no empty cell remained
```

```
            solutions_.push_back(grid);             // record a full solution
            return;
        }

        for (int digit = 1; digit <= kSize; ++digit) {          // try candidates
            if ((bestMask & bitFor(digit)) == 0) continue;      // not a candidate
            grid.setValue(bestRow, bestCol, digit);             // tentative place
            tracker.markUsed(bestRow, bestCol, digit);          // update masks
            search(grid, tracker);                              // recurse deeper
            tracker.markFree(bestRow, bestCol, digit);          // undo masks
            grid.setValue(bestRow, bestCol, 0);                 // undo the cell
            if (limitReached()) {                               // stop if capped
                hitLimit_ = true;
                return;
            }
        }
    }

    SudokuGrid puzzle_;                 // the puzzle being solved
    std::size_t limit_;                 // 0 = unlimited solutions
    std::vector<SudokuGrid> solutions_;  // every solution discovered
    std::size_t nodesVisited_;          // search-tree nodes expanded
    bool hitLimit_;                     // true if the cap ended the search
};

/*****************************************************************************
 * Class: SudokuApplication
 * Purpose: Drives the program: loads each puzzle file, prints the required
 *          report (file name, puzzle, solutions, count), and times the search.
 * Authorship: <your name>, generated with <GenAI name> and revised by hand.
 *****************************************************************************/
class SudokuApplication {
public:
    SudokuApplication(const std::vector<std::string>& puzzleFiles,
                      std::size_t solutionLimit)
        : puzzleFiles_(puzzleFiles), solutionLimit_(solutionLimit) {}

    // Process every requested puzzle file in order.
    void run() {
        for (std::size_t i = 0; i < puzzleFiles_.size(); ++i) {
            processPuzzle(puzzleFiles_[i]);         // handle one file
        }
    }

private:
    // Produce the complete report for a single puzzle file.
    void processPuzzle(const std::string& fileName) {
        std::cout << "\n================================================\n";
        std::cout << "Puzzle file: " << fileName << '\n';   // required heading
```

```
        std::cout << "================================================\n";

        SudokuGrid puzzle;                          // empty grid object
        if (!puzzle.loadFromFile(fileName)) {       // read the file
            std::cout << "ERROR: could not open or parse '" << fileName << "'\n";
            return;                                 // skip to next puzzle
        }

        std::cout << "\nPuzzle:\n";                  // label the puzzle
        puzzle.print(std::cout);                    // show it as loaded

        const std::chrono::steady_clock::time_point start =
            std::chrono::steady_clock::now();       // start the clock
        SudokuSolver solver(puzzle, solutionLimit_);    // build the solver
        solver.solve();                             // run the recursion
        const std::chrono::steady_clock::time_point stop =
            std::chrono::steady_clock::now();       // stop the clock

        const std::vector<SudokuGrid>& found = solver.solutions();
        if (found.empty()) {                        // unsatisfiable puzzle
            std::cout << "\nNo solution found\n";
        } else {                                    // print every solution
            for (std::size_t i = 0; i < found.size(); ++i) {
                std::cout << "\nSolution " << (i + 1) << ":\n";
                found[i].print(std::cout);          // nine rows of digits
            }
        }

        std::cout << "\nSolutions found: " << found.size();
        if (solver.hitLimit()) {                    // cap was reached
            std::cout << " (search stopped at the requested limit)";
        }
        std::cout << '\n';

        const double elapsedMs =                    // milliseconds elapsed
            std::chrono::duration<double, std::milli>(stop - start).count();
        std::cout << "Search nodes visited: " << solver.nodesVisited() << '\n';
        std::cout << "Elapsed time: " << elapsedMs << " ms\n";
    }

    std::vector<std::string> puzzleFiles_;   // files to process
    std::size_t solutionLimit_;              // 0 = report all solutions
};

/*****************************************************************************
 * Function: main
 * Purpose : Parse optional command-line arguments, default to the five
 *           assignment puzzle files, and run the application.
 * Authorship: <your name>, generated with <GenAI name> and revised by hand.
```

```
 ********************************************************************************/
int main(int argc, char* argv[]) {
    std::vector<std::string> files;          // puzzle files to solve
    std::size_t solutionLimit = 0;           // 0 means "no limit"

    for (int i = 1; i < argc; ++i) {         // inspect each argument
        const std::string arg(argv[i]);      // current argument as a string
        if (arg.rfind("--max=", 0) == 0) {   // "--max=N" caps solutions
            try {
                solutionLimit = static_cast<std::size_t>(
                    std::stoull(arg.substr(6)));   // parse N
            } catch (const std::exception&) {      // bad number supplied
                std::cerr << "Ignoring invalid argument: " << arg << '\n';
                solutionLimit = 0;                 // fall back to unlimited
            }
        } else {
            files.push_back(arg);            // any other argument is a file
        }
    }

    if (files.empty()) {                     // no files given: use defaults
        for (int i = 1; i <= 5; ++i) {
            files.push_back("puzzle" + std::to_string(i) + ".txt");
        }
    }

    SudokuApplication application(files, solutionLimit); // build the app
    application.run();                                   // do the work
    return 0;                                    // normal exit
}
```

e) Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
   1) Correctness
      a) Both codes run correctly, showing the correct outputs for the given puzzles.
      b) In Lumo's markFree definition, it flips all the bits and removes all bits except bit 1, which works for bits 1-9, but it would be better and more concise to simply remove the single target byte
   2) Execution time:
      a) Lumo's code stores the solution every time it obtains one, which if it were on a much larger puzzle, would slow it down immensely
   3) Space complexity:
      a) Since the puzzle is at a fixed 9×9, space complexity should be O(1) for both codes, however, Lumo's code stores the solution, then outputs it, while Leo's code outputs the solution immediately then discards it, rather than taking up excess memory.
   4) Maintainability:

a) Lumo's code is much much longer than Leo's code by more than 200 lines, but this does include comment lines from Lumo, which is explains more than Leo's code does. Their output codes for the finished puzzles both make me wish that it was slightly easier to read.

f) Select one to use as the basis for this assignment and justify why you picked it.
   1) While I like that Lumo's code includes time taken to complete the puzzle and how many search nodes visited, Leo's code is much more manageable in terms of length and how much space is stored.

g) Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
   1) Just for clarity, I wanted to add a space between the original puzzle clues and the solution
   2) The program will now check if there's contradictory clues rather than checking through all the solutions regardless and will automatically give no solution, improves correctness and execution time
   3) Added more comment lines